

Task 1

Repository Foundation

A tightly scoped manual-and-Codex execution guide for initializing the fraud-detection project repository without starting backend, frontend, database, streaming, ML, or AI implementation.

Prepared for: Ritwik Dhull

Version: 1.0

Date: 9 August 2026

Scope: Repository foundation only

Operating rule: Codex pauses for manual blockers and reports pending manual actions



1. Task Definition

ONLY OUTCOME FOR TASK 1

Create a clean, documented and agent-ready repository foundation. No application features, APIs, databases, Redis workers, frontend screens, fraud rules, machine learning, AI, scoring, or model code may be implemented.

1.1 Task boundary

Included now	Explicitly excluded now
Empty GitHub repository and local clone	FastAPI endpoints or business logic
Planning documents under `docs/planning/`	React pages or components
Root `AGENTS.md` and working agreements	PostgreSQL, Redis, Docker services or workers
README, `.gitignore`, `.env.example`	Transaction contracts and database schemas
Approved repository directory skeleton	Fraud scoring, rules, ML, AI or dataset processing
Verification and handoff conventions	Production dependencies or deployment

1.2 Task lifecycle

Stage	Owner	Result
A. Manual preflight	Ritwik / human owner	Repository, local clone and planning files are available
B. Codex inspection	Codex	Read-only repository plan and prerequisite check
C. Manual approval	Ritwik / human owner	Plan accepted or corrected
D. Codex implementation	Codex	Repository foundation files and directories created
E. Manual review	Ritwik / human owner	Diff reviewed; commit/push approved
F. Codex handoff	Codex	Verification plus pending manual actions

STOP RULE

If Codex needs a manual installation, login, repository decision, permission, missing file, external account action, or owner approval, it must stop the affected work and request that action precisely. It must not assume completion or continue around the blocker.

2. Manual Part - Before Codex Starts

Complete only the applicable actions below. If you have already completed an item, Codex should verify it rather than asking you to repeat it.

#	Manual action	Expected evidence	Blocking?
M1	Create an empty GitHub repository named `adaptive-fraud-detection-platform`	Repository URL	Yes
M2	Clone the repository to the development machine	Local path and `.git/` present	Yes
M3	Create `docs/planning/` and place the approved planning PDFs, patent draft and architecture diagram inside	Files visible to Codex	Yes
M4	Open Codex from the repository root	Codex confirms expected working directory	Yes
M5	Set normal workspace permissions; do not grant unrelated filesystem access	Codex can read/write inside repository	Yes
M6	Confirm team member names and repository invitations	Access list	No - may remain pending
M7	Download the ULB CSV and retain licence/source details	Local file and source record	No - not needed in Task 1

2.1 Suggested manual commands

```
Clone and enter repository
git clone <repository-url>
cd adaptive-fraud-detection-platform
```

```
Create planning directory
mkdir -p docs/planning
```

```
Confirm repository root
git rev-parse --show-toplevel
git status
```

WINDOWS NOTE

You may create `docs/planning/` through File Explorer instead of the terminal. The important requirement is that Codex is launched from the repository root and can see the planning files.

2.2 Manual safety rules

- Do not paste GitHub passwords, access tokens, private keys or unrelated credentials into the Codex prompt.
- Do not place `creditcard.csv`, `.env`, real secrets, local caches or generated artefacts in the repository.
- Do not ask Codex to push or merge until you have reviewed the generated diff.
- If a required system-level tool is missing, install or approve it only after Codex explains why Task 1 requires it. Task 1 should normally require only Git and Codex.

3. Codex Part A - Plan and Preflight

Paste the following into Codex from the repository root. Use Plan mode. Codex must remain read-only until you approve its proposed foundation.

TASK

We are performing only Task 1: repository foundation for the adaptive fraud-detection platform. Machine learning and AI are deferred.

READ FIRST

Inspect the repository and every file under `docs/planning/`. Confirm the repository root and current Git status.

PLAN ONLY

Do not create, modify, install or delete anything yet. Propose the minimal foundation covering:

1. Root `AGENTS.md`.
2. `README.md`, `.gitignore` and `.env.example`.
3. Backend, frontend, packages, infrastructure, tests, documentation, scripts and data directory placeholders.
4. Python and Node dependency-management approach for later tasks, without installing dependencies now.
5. Future Docker Compose location, without creating services now.
6. Formatting, linting, typing and testing conventions for later implementation.
7. One future unified verification command.
8. Files, datasets, secrets and artefacts that must never enter Git.

EXCLUSIONS

Do not implement APIs, databases, Redis, Docker services, UI features, transaction logic, fraud rules, model code, ML, AI, scoring or dataset processing.

MANUAL-BLOCKER PROTOCOL

If any prerequisite requires a human action, stop the affected work and ask using the exact `MANUAL ACTION REQUIRED` format defined below. Do not assume the action is complete. Resume only after I confirm completion.

RETURN

Exact proposed tree, files to create, contents/purpose of each root file, validation approach, unresolved questions and all currently pending manual actions. Wait for approval before editing.

3.1 Expected Codex plan

- A minimal directory tree with no premature application code.
- A concise `AGENTS.md` that points to detailed planning documents rather than duplicating them.
- A `.gitignore` policy covering raw data, secrets, model artefacts, caches, logs, IDE files and local environment files.
- A clear statement that framework dependencies and Docker services are deferred to the next technical task.
- Verification steps that inspect the tree, Git exclusions and instruction files without pretending the application exists.
- A final `Pending Manual Actions` section, even when its value is `None`.

4. Mandatory Manual-Blocker Protocol

CODEX MUST ASK - NOT GUESS

This protocol applies whenever Codex cannot proceed safely without a human action. Codex may continue independent, unblocked inspection, but it must not bypass, simulate or silently replace the missing human step.

4.1 Required request format

MANUAL ACTION REQUIRED - [short title]

Status: BLOCKING or NON-BLOCKING

Why it is needed: [one precise explanation]

What I verified: [evidence Codex already checked]

Your exact action: [numbered steps or exact UI path/command]

Expected confirmation: [what the user should reply or provide]

Safety note: [credentials, destructive action or permission warning if relevant]

Codex state: Paused before [specific operation]. I will resume from [specific step] after confirmation.

Reply with: `Done - [requested confirmation]` or describe the problem you encountered.

4.2 Actions that trigger this protocol

Trigger	Codex behaviour
Repository does not exist or wrong folder is open	Stop and ask the owner to create/open the correct repository
Planning file is missing	Name the missing file and ask the owner to attach/copy it
Git login, remote access or invitation is required	Ask the owner to authenticate or grant access outside the prompt
System-level installation or admin permission is required	Explain the need and ask the owner to install/approve it
A secret or credential is required	Ask the owner to configure it securely; never request the secret value in chat
A destructive or irreversible operation is proposed	Stop, identify the exact target and request explicit approval
A product, architecture or ownership decision is unresolved	Present bounded options and wait for the owner decision
Commit, push, branch protection or merge is needed	Ask for approval or for the owner to perform the operation

4.3 What Codex must not do

- Do not continue by inventing a repository URL, path, file, team member, licence, credential or approval.
- Do not ask for passwords, tokens, private keys or secret values in the conversation.
- Do not install extra production dependencies merely to complete repository scaffolding.
- Do not broaden Task 1 into application implementation.
- Do not call a manual task complete because it appears likely or because a placeholder was created.

5. Manual Approval Before Codex Edits

After Codex returns its read-only plan, review it using this checklist. Send corrections if any item fails.

Check	Approval condition
Scope	Only repository foundation is proposed
ML/AI boundary	No ML, AI, scoring, data science or fraud logic
Application boundary	No API, database, Redis, Docker service or frontend feature implementation
Instructions	`AGENTS.md` contains the manual-blocker and pending-manual-actions rules
Data safety	`data/raw/`, `.env`, secrets, artefacts and logs are excluded from Git
Simplicity	No unnecessary framework, tool or production dependency is introduced
Verification	Codex can verify tree, instruction files and Git ignore behaviour
Handoff	Codex promises exact verification results and pending manual actions

5.1 Approval response

If the plan is correct, reply:
`Approved. Implement Task 1 exactly as planned. Do not begin the next technical task.`

If changes are required, reply:
`Revise the Task 1 plan with these corrections: [list]. Remain in Plan mode and wait again.`

5.2 Codex implementation scope after approval

File / area	Purpose
`AGENTS.md`	Durable scope, safety, review, blocker and handoff rules
`README.md`	Mission, current phase, prerequisites and navigation
`.gitignore`	Exclude raw data, secrets, caches, logs, builds and artefacts
`.env.example`	Names/placeholders only; no real values
Directory skeleton	Approved ownership boundaries with small placeholder READMEs where Git needs tracked files
`docs/planning/README.md`	Identify authoritative planning documents and their purpose
Verification script or documented commands	Confirm structure, exclusions and instruction discovery

6. Codex Part B - Implementation Prompt

Use this only after approving the Codex plan. It authorizes the repository-foundation changes and nothing beyond them.

AUTHORIZATION

Implement Task 1 exactly according to the approved repository-foundation plan.

REQUIRED BEHAVIOUR

- Inspect before editing and preserve all existing user files.
- Create only the approved foundation files and directories.
- Add the manual-blocker protocol and final pending-manual-actions requirement to root `AGENTS.md`.
- Do not start the next technical task.
- Do not install dependencies or initialize application frameworks unless they were explicitly approved in this Task 1 plan.
- Do not commit, push or merge without separate approval.

VERIFY

- Show the final repository tree.
- Verify the repository root and Git status.
- Verify raw data, `.env`, secrets, caches, logs and artefacts are ignored.
- Verify `AGENTS.md` contains the scope, blocker and handoff rules.
- Review the diff for accidental implementation or unrelated changes.

IF BLOCKED

Use the exact `MANUAL ACTION REQUIRED` format and wait for confirmation before the blocked operation.

FINAL HANDOFF

Return the required Task 1 completion report defined in this guide. Every handoff must end with `Pending Manual Actions`, divided into blocking and non-blocking items. Use `None` explicitly where appropriate.

6.1 Task 1 exit criteria

- Codex is operating from the correct Git repository root.
- Planning documents are discoverable under `docs/planning/`.
- Root `AGENTS.md`, `README.md`, `.gitignore` and `.env.example` exist and match the approved scope.
- The approved directory skeleton exists without premature application code.
- Sensitive, local and generated files are excluded from Git.
- No ML/AI, API, database, streaming, UI or fraud logic was implemented.
- Codex reports exact verification results, residual risks and pending manual actions.
- No commit, push or next task begins without owner approval.

7. Required Codex Final Handoff

Codex must use this structure after implementing Task 1. This format makes manual responsibilities visible instead of burying them in the completion summary.

TASK 1 STATUS
`COMPLETED` | `PARTIALLY COMPLETED` | `BLOCKED`

Outcome
[One sentence describing the repository foundation now present.]

Files created or changed
[Grouped list with purpose.]

Verification performed
[Exact commands/checks and results. Never claim an unrun check.]

Scope confirmation

- No application implementation started.
- No ML/AI work started.
- No unapproved dependency installed.
- No commit/push/merge performed unless separately approved.

Risks or deviations
[List or `None`.]

PENDING MANUAL ACTIONS - BLOCKING
[Exact action, reason, evidence needed and resume point; or `None`.]

PENDING MANUAL ACTIONS - NON-BLOCKING
[Review, repository access, commit/push, team or future dataset actions; or `None`.]

Next technical task
[Name only. State clearly that it has not started.]

7.1 Expected manual actions after Codex finishes

Manual action	When required
Review every changed file and the final Git diff	Always
Confirm no secrets, raw dataset or unrelated file is staged	Always
Approve, create or amend the initial commit	After successful review
Push the approved commit to GitHub	After commit approval
Invite team members and configure branch protection	Before shared development
Resolve any item under `PENDING MANUAL ACTIONS - BLOCKING`	Before the next task
Keep non-blocking items in the project ledger	Before their stated deadline

TASK 1 ENDS HERE

After the repository foundation is reviewed and committed, stop. The next technical task must receive a separate scope and approval. No backend, frontend, database, streaming, ML or AI work is implicitly authorized by this PDF.

8. Official Codex Guidance Basis

This workflow follows official OpenAI documentation: keep durable repository rules in `AGENTS.md`, use Plan mode before multi-step edits, give Codex explicit behaviour, context, constraints and verification requirements, maintain tight permissions, and require testing/review before accepting changes.

- Codex best practices: <https://learn.chatgpt.com/guides/best-practices>
- AGENTS.md instructions: <https://learn.chatgpt.com/docs/agent-configuration/agents-md>
- Prompting Codex: <https://learn.chatgpt.com/docs/prompting>

OWNER-CONTROL PRINCIPLE

Codex performs the maximum safe repository work. The human owner retains account access, credentials, permissions, plan approval, diff review, commit/push authority and the decision to begin the next task.